汽车零部件理论计算平台 — 软件开发规则

> 适用范围：本仓库（`e:\TRAECode\CTR_CODE`）全部前端 / Electron / 测试代码的新增与修改。 配套文档：《公式编写规则》（同目录），新增公式时两份文档一起读。

---

1. 技术栈与运行环境

项	版本/约定	说明
框架	Electron 32 + React 18 + Vite 5	桌面单机应用，离线使用
语言	TypeScript 5.6（严格模式）	主进程文件为 `.cjs`（CommonJS），渲染层为 ESM
测试	Vitest 2	`npm test`（= `vitest run`）
打包	electron-builder 25（NSIS）	配置在 `electron-builder.config.cjs`
主要依赖	echarts（图表）、katex（公式）、docx/exceljs/html2canvas（报告导出）、bcryptjs（口令）、pdfjs-dist（PDF 阅读）	新增依赖前先确认现有依赖不能覆盖需求

常用命令（`package.json` scripts）：

```
npm run dev        # 开发：Vite(5173) + Electron 同时启动
npm run dev:web    # 仅浏览器调试页面
npm run typecheck  # tsc --noEmit，提交前必过
npm test           # vitest run，提交前必过
npm run build      # tsc + vite build + electron-builder 出安装包
```

验证链（每次提交前必须全绿）：`npm run typecheck` → `npm test`。

---

2. 工程目录结构

```
CTR_CODE/
├── electron/              # 主进程（CommonJS）
│   ├── main.cjs           # 窗口、IPC、菜单
│   ├── preload.cjs        # contextBridge 暴露的桥接 API
│   ├── store.cjs          # 本地持久化（JSON 文件存储）
│   └── license.cjs        # 授权逻辑
├── src/
│   ├── formula/           # 公式核心层（与界面解耦）
│   │   ├── types.ts       # FormulaSpec / ParamSpec / OutputSpec 等类型定义
│   │   ├── units.ts       # 单位分组与 SI 换算（toSI / fromSI / autoUnit / formatNumber）
│   │   └── registry.ts    # 公式注册表（新增公式唯一需要改动的基础设施文件）
│   ├── domains/           # 公式实现，按领域分目录
│   │   ├── fluid/         # 流体设计
│   │   ├── structure/     # 结构设计
│   │   └── acoustics/     # NVH 设计
│   ├── components/        # 计算页组件：ParamForm / FormulaView / ResultPanel / CurveChart
/ SensitivityChart
│   ├── pages/             # 页面：LibraryPage、CalculatorPage、HomePage、ProjectsPage、
HistoryPage 等
│   ├── report/            # 计算书 / 试验报告生成（buildReport.ts、testReport.ts、
exportReport.ts、db.ts）
│   ├── api.ts             # 渲染进程统一访问层（封装 preload 桥接）
│   └── styles.css         # 全局样式
├── tests/                 # Vitest：formulas.test.ts / units.test.ts / testReport 相关
├── build/                 # 打包资源（icon.ico 等）
├── 公式梳理/              # 公式理论来源文档（参考材料，不打包）
├── .tmp/                  # 临时脚本（如取 benchmark 期望值的测试），用后清理
└── electron-builder.config.cjs
```

分层铁律：`src/domains/`（公式实现）只依赖 `src/formula/`（类型与单位），**不允许反向依赖** `pages/components/report`。公式层必须可以在无界面环境下被 vitest 直接调用。

---

3. 核心架构原则

3.1 元数据驱动

界面不写死任何公式。计算页、公式库、搜索、历史记录全部由 `FormulaSpec` 元数据自动渲染：

- 新增公式 = 新建一个 `FormulaSpec` 对象 + 在 `registry.ts` 注册，**不需要改任何界面代码。**
- 输入表单由 `inputs` 自动生成（含单位下拉、范围提示、条件显示）。
- 结果区由 `outputs` 自动生成（含分组、主结果高亮、自动选单位）。
- 公式说明由 `latex`（KaTeX）与 `assumptions` 自动展示。

3.2 SI 单位内部制

- `solve` 的输入、输出**一律 SI 基本/导出单位**（m、Pa、kg、K、Hz、W ...）。
- 界面显示值与 SI 值的换算只在 `CalculatorPage` 入口（`toSI`）和结果显示（`autoUnit`）发生，公式内部**禁止出现 mm、kPa、℃ 等工程单位换算代码。**
- 例外：`temperature` 组是平移换算（℃ ↔ K），由 `toSI/fromSI` 特例处理；温差用 `temperatureDelta` 组（K 与 ℃ 等值，不偏移）。

3.3 单位按参数独立记录

- 每个输入项的显示单位按**输入 key** 各自记录（`units[p.key]`），**禁止按 unitGroup 共享。**（历史教训：按组共享时，同组不同参数（如 100 Pa 声压与 100 kPa 工作压力）互相污染，导致计算结果全空。）
- 温度单位切换时只对**该参数**做 ℃↔K 平移，不得影响其它参数。
- 读取历史数据时注意兼容早期按组记录的格式（`buildReport.ts` 已做兼容，新代码沿用同一约定）。

3.4 无解与异常约定

- `solve` 是纯函数：同样的输入永远得到同样的输出，无副作用、不读写文件、不依赖界面状态。
- 输入非法 / 物理上无可行解时：**返回所有 key 均为 NaN 的对象**，或直接抛异常。界面已 try/catch，结果为 NaN 时显示为空（`formatNumber` 输出 `–`）。
- 不要在 `solve` 里 `alert` / `console.log` 弹错，错误信息通过返回 NaN 或 `assumptions` 说明传达。

---

4. 界面与组件约定

- 计算页三栏：左 `ParamForm`（输入）、中 `FormulaView`（公式与说明）、右 `ResultPanel` + `CurveChart` + `SensitivityChart`（结果、曲线、敏感性）。三栏宽度可拖动并持久化到 localStorage（键 `ctr.calcLeftWidth` / `ctr.calcRightWidth`）。
- 条件显示：输入和输出都支持 `showIf: { key, min?, max? }`，用于「按腔数逐腔展开」这类场景。三处必须一致处理：表单（`ParamForm`）、结果区（`ResultPanel`）、CSV 导出（`CalculatorPage`）。
- 结果展示顺序固定：**结果 → 结果曲线 → 参数敏感性分析**（敏感性永远在右栏最后）。
- 结果自动选单位：有 `unitGroup` 的输出用 `autoUnit`（优先工程习惯单位，量级越界自动缩放）；无 `unitGroup` 的原样显示 `unit` 字段。
- 新增页面：在 `src/pages/` 建组件，在 `App.tsx` 登记路由/入口；样式优先复用 `styles.css` 既有类名，避免大面积新增样式。

5. 数据与持久化约定

- 渲染进程一律通过 `src/api.ts` 访问主进程能力，**不直接 import electron**。
- 历史记录保存内容：`{ values, units, inputsSI }` + `outputs`（SI），报告与复算以 SI 为准。
- 本地数据由 `electron/store.cjs` 管理（用户目录 JSON），离线单机，无网络依赖。
- 安装包不含源码与开发依赖（`asar: true`）；`pdfjs-dist` 因只有 ESM 构建需 `asarUnpack`。

---

6. 测试规则

1. 测试全部放 `tests/`，Vitest 运行，**不依赖 Electron、不依赖界面**。
2. `tests/formulas.test.ts` 自动遍历注册表全部公式做基准回归——新增公式一旦注册即被覆盖，无需改测试文件。
3. 公式的 `benchmarks` 是正确性的最终防线：
   - `input` / `expect` 一律 **SI 单位**；
   - `tolerance` 一般取 `1e-9`（相对容差），物理模型类可放宽但须注明理由；
   - **`expect` 数值禁止手算填写。**标准流程：在 `.tmp/` 建临时 vitest 文件，实跑 `solve` 打印 `JSON.stringify(结果)`，把输出原样抄入 `benchmarks`，再人工核对量级与物理合理性，最后删除临时文件。
4. 修改已有公式的 `solve` 后，若结果变化是预期内的（如修正了错误公式），按第 3 条流程**重新实跑取数**更新 `expect`，并在提交说明中写明原因。
5. 单位换算改动必须同步维护 `tests/units.test.ts`。

7. 构建与打包规则

```
npm run build    # 完整产出：release/Theoretical Calculation Platform Setup <version>.exe
```

- 产物为 NSIS 安装包（非一键安装，允许用户改安装目录，桌面/开始菜单快捷方式名「汽车零部件理论计算平台」）。
- **打包前必须结束正在运行的应用进程**（尤其是 `release\win-unpacked\` 下启动的实例），否则 `icudtl.dat` 等文件被占用导致打包失败。
- **不要并行执行两个** `npm run build`，会互相清目录。
- `signAndEditExecutable` 当前为 `false`：该步骤会下载 winCodeSign 工具包，在无管理员权限/未开开发者模式的机器上解压失败。有签名证书后可在 `electron-builder.config.cjs` 改回。
- 发布版本时同步修改 `package.json` 的 `version`（安装包文件名含版本号）。

8. Git 提交规范

- 提交信息用中文，Conventional Commits 风格：`feat(structure): …`、`fix(ui): …`、`test(formulas): …`、`docs: …`。
- PowerShell 无 heredoc，多行提交说明用多个 `-m`：`git commit -m "标题" -m "正文说明"`
- 一次提交只做一件事，只 `git add` 与本任务直接相关的文件，不用 `git add -A`。
- 提交前确认 `npm run typecheck` 与 `npm test` 全绿。
- 不要把 `release/`、`dist/`、`node_modules/`、`.tmp/` 提交进仓库。

9. 常见坑速查

现象	原因	处理
默认输入下结果全空（—）	单位换算错位（如 Pa 当 kPa）或输入物理上无可行解	检查各输入的单位下拉是否按 key 独立；检查 `solve` 的合法性门槛
℃ 切 K 后数值错误	温度是平移换算，不能乘倍率	只能用 `toSI/fromSI` 的 temperature 特例，禁止 `value * factor`
打包失败、文件被占用	旧实例在跑	结束 `win-unpacked` 下的应用进程后重打
新增公式界面里看不到	忘了在 `registry.ts` 注册	import 并加入 `ALL_FORMULAS`
benchmark 偶尔挂	`expect` 是手算/约等值	按第 6.3 条流程实跑取全精度值
曲线图不出	`curve.compute` 返回空或 x 未升序	compute 必须返回 x 严格递增的点列

本规则供评审修改，条款后注明「历史教训」的条目均对应真实踩过的坑，修改前请先理解当时的原因。